01-technical-requirements-and-design

01 — Technical Requirements and Design

Project: LDAPA Intelligent Portal **Client:** Learning Disabilities Association of Pennsylvania (LDA of PA) **Status:** Deployed to production on Railway + Vercel **Date:** April 2026

---

1. Purpose

The LDAPA Intelligent Portal is a conversational web application that helps Pennsylvanians find learning-disability support providers — tutors, therapists, psychologists, lawyers, schools, and educational advocates — through a natural-language chat instead of a traditional search form. The system also gives LDA of PA staff an admin panel for managing the provider directory, reviewing chat transcripts, and tracking usage.

It replaces (or augments) the static directory listing previously maintained by Brilliant Directories. The entire initial provider dataset (≈3,600 members) was imported from the Brilliant Directories CSV export.

---

2. Functional Requirements

2.1 Public-facing portal (visitors / parents / caregivers / adults with LDs)

#	Requirement	How it is met
F-1	Users can describe what they need in plain English (“I need a reading tutor near Pittsburgh for my 9-year-old”)	Chat UI on `/chat`; backend converts message → structured filters via LLM
F-2	System returns a short list of matching providers with enough info to contact them	Provider cards rendered from backend response; show name, type, city, phone, email, website, credentials, pricing, insurance
F-3	System answers general informational questions (“What is an IEP?”) without forcing a search	LLM detects general-question intent and answers directly; no DB query runs
F-4	System asks clarifying questions when the request is too vague	LLM returns `needs_more_info: true` for early turns with no useful filters
F-5	System escalates crisis messages (self-harm, abuse, emergency) to hotlines instead of attempting to help itself	LLM sets `escalate: true`; response directs to 988 / 911 / LDA of PA

#	Requirement	How it is met
F-6	System recovers gracefully when no exact match is found	Multi-pass broadening: geo → strict → relax age → relax city → state-wide
F-7	Users can give thumbs-up/down feedback on any assistant message	`POST /api/feedback` stores the rating in `chat_feedback`
F-8	Conversations persist across page reloads within a session	Session ID is returned by the backend and echoed on subsequent requests
F-9	Responses render markdown formatting (bold, bullet lists)	Frontend uses `react-markdown`

2.2 Admin panel (LDA of PA staff)

#	Requirement	How it is met
A-1	Staff log in with email + password	`POST /api/admin/login` → JWT token stored in browser localStorage
A-2	Staff see a dashboard of usage and feedback	`/overview` page — total providers, verified/unverified counts, chat volume by day, top conversation themes, average feedback rating
A-3	Staff can list, search, filter, and paginate providers	`/providers` page with search, status filter, profession filter
A-4	Staff can create, edit, and soft-delete providers	Full CRUD forms; soft delete flips `is_deleted = 1` (records are never physically deleted)
A-5	Staff can mark providers verified or archived, individually or in bulk	Verify / archive buttons + multi-select bulk actions
A-6	Staff can import additional providers from the Brilliant Directories CSV export	Three-step wizard at `/import-export`: upload → preview (validation errors/warnings + first N rows) → confirm
A-7	Staff can review every chat session and see the messages + feedback each received	`/chats` list + session detail view
A-8	Staff can see which providers were shown in any given conversation	`providers_shown` JSON column on each assistant message

2.3 Non-functional requirements

#	Requirement	How it is met
N-1	Response time under 5 seconds for a chat turn	Two LLM calls (`gpt-5-mini` , reasoning effort `low`) plus one DB query; typical end-to-end 2–4 s

#	Requirement	How it is met
N-2	Frontends available globally with low latency	Vercel's edge CDN serves the Next.js apps
N-3	Backend must survive the OpenAI API being temporarily unavailable	Hard fallback path: keyword-regex filter extraction + templated responses
N-4	System must be operable by non-technical staff	Admin panel is the only interface staff interact with; no shell access or SQL required
N-5	User data is not shared with third parties beyond what the LLM call requires	No analytics SDK, no tracking pixels; OpenAI is the only external service that receives chat content
N-6	Rate-limiting to prevent abuse of the free LLM budget	In-memory sliding window: 30 requests / 60 seconds per session
N-7	Data is durable across deploys	PostgreSQL runs as a separate Railway service with its own volume; it is not redeployed when the backend redeploys
N-8	Secrets are never committed to git	All credentials live in Railway / Vercel environment variables
N-9	Admin routes cannot be accessed without authentication	JWT middleware (`require_admin`) guards every `/api/admin/*` endpoint

---

3. Design Principles

These are the design decisions that shaped the codebase. Whoever inherits the project should understand the "why" before changing any of them.

3.1 Two LLM calls per chat turn, not one

A single LLM call would have to both (a) understand what the user wants AND (b) decide which providers to recommend. That couples search logic to the model's prompt, makes it hard to debug, and costs more tokens.

Instead, the pipeline is:

1. **Call 1 — Filter Extraction.** Read the full conversation, emit JSON describing intent (profession type, location, age group, insurance, etc.).
2. **Database query.** Run the structured search — fast, deterministic, auditable.
3. **Call 2 — Response Generation.** Given the conversation + the rows returned by the query, write a friendly answer.

This separation means search quality is improvable *without* retraining or reprompting the writer persona, and vice versa.

3.2 Tiered search broadening over “no results”

Parents looking for help rarely want “0 results” — they want *something*. So the search engine tries progressively broader queries: geo-proximity (if a ZIP is known) → strict city + age → relax age → relax city → state-wide. When broadening kicks in, the system honestly tells the user “these aren’t exact matches, but they’re the closest we found.”

3.3 Lawyers de-prioritized unless explicitly asked for

The directory has 792 lawyers (22% of the data). If a parent just asks for “help with my child’s reading,” ranking lawyers equally would flood the results with legal advice the user didn’t ask for. An `ORDER BY CASE WHEN profession = 'Lawyer' THEN 1 ELSE 0 END` penalty fixes that — lawyers still appear when the user mentions legal intent.

3.4 Keyword fallback for LLM downtime

If `OPENAI_API_KEY` is missing or the OpenAI call raises, the backend falls back to a regex/keyword extractor and a four-branch response template. Users get a less-smart but still functional experience. This is important because it means **the portal never goes dark purely because of an OpenAI outage** — it just becomes a classical search form.

3.5 Dual-mode database (SQLite dev, PostgreSQL production)

A single database wrapper (`backend/app/database.py`) speaks either SQLite (local dev, via `aiosqlite`) or PostgreSQL (production, via `asyncpg`) based on the `DATABASE_URL` environment variable. SQL differences (`?` vs `$1` placeholders, `datetime('now')` vs `CURRENT_TIMESTAMP`) are rewritten automatically at query time. This means developers can work locally with a `.db` file and deploy to a managed Postgres without any code changes.

3.6 Soft deletes, not hard deletes

Providers are never physically removed from the database. Deleting a provider just flips `is_deleted = 1` . This is important because the directory data is costly to reconstruct, and staff occasionally need to un-delete records.

3.7 Anonymous chat sessions

The public portal does not ask users to register. Every chat session gets a random UUID. The only identifying info stored is an optional location (city / ZIP) the user mentioned themselves — and that’s kept so the dashboard can show “most chats come from Philadelphia.”

3.8 Integer booleans

All boolean columns (`sliding_scale` , `ld_adhd_specialty` , etc.) are stored as `INTEGER` 0/1 instead of `BOOLEAN` . This avoids SQLite’s lack of a native boolean type and keeps the schema identical between SQLite and PostgreSQL. The backend converts them to Python booleans on the way out.

4. Constraints

- **Provider data is read-only from the user's perspective.** The chat interface does not allow anonymous users to add, edit, or flag providers — only LDA of PA staff can do that via the admin panel.
- **Geographic scope is Pennsylvania.** The search engine assumes PA; `state_code` defaults to `PA` and distance math uses PA latitude (~40°N) for the miles-per-degree constants. Expanding to other states would require revisiting both.
- **Single admin role.** There is one role — "admin" — not granular permissions. Anyone with the admin password can do everything.
- **In-memory rate limiting.** The 30-req/60-s limiter lives in process memory. If the backend is ever scaled to multiple instances, it will need to move to Redis or a similar shared store.
- **OpenAI is the only LLM provider.** The system uses OpenAI's Responses API with `gpt-5-mini`. Swapping providers (Anthropic, etc.) would require replacing `backend/app/services/llm.py`.

5. Out of Scope (Intentionally)

These were considered and deliberately not built:

- **Provider self-service.** Providers cannot log in to update their own listings. All edits go through LDA of PA staff.
- **Appointment booking.** The portal shows contact info (phone, email, website) but does not book appointments.
- **Payment or billing.** The directory is free to use; no commerce layer exists.
- **Multi-language.** English only for the MVP.
- **Provider reviews / ratings from users.** Feedback exists on *chat responses*, not on *providers*. Staff verification is the quality signal.
- **SMS / voice / email bot.** Web chat only.
- **A second admin role (e.g., "viewer").** One level of access only.

6. Technology Choices (and Why)

Layer	Choice	Why
Backend framework	FastAPI	Async, auto-generated OpenAPI docs at `/docs`, Pydantic validation, fast
Language	Python 3.11	Best match for the OpenAI SDK and

Layer	Choice	Why
		Pydantic; Railway has native buildpack support
Database (dev)	SQLite via `aiosqlite`	Zero-config for local work; a `.db` file is the whole database
Database (prod)	PostgreSQL via `asyncpg` + Railway managed plugin	Managed backups, WAL, connection pooling; Railway injects `DATABASE_URL` automatically
LLM	OpenAI `gpt-5-mini` (Responses API, reasoning effort `low`)	Cheap per token, fast, and "reasoning low" is enough for structured filter extraction + short replies
Frontend framework	Next.js 16 + React 19	App router, SSR, Vercel-native deployment
Styling	Tailwind CSS 4	Utility-first; no custom CSS burden
Authentication	Bcrypt-hashed passwords + HS256 JWT	Simple, stateless, no session store needed
Hosting (frontend)	Vercel	Zero-config Next.js deploys, free tier, edge CDN
Hosting (backend)	Railway	Python buildpack, managed Postgres, single dashboard
Diagrams	Mermaid	Text-based, version-controlled alongside the code

---

7. Success Criteria

The system is considered successful if:

1. A parent with no technical background can describe their situation in their own words and leave with the names and contact details of 1–5 relevant PA providers.
2. LDA of PA staff can keep the directory up to date without developer involvement (bulk CSV import + individual edits in the admin panel).

3. Staff can see, at a glance, how many people are using the portal, what they're asking about, and whether the answers were helpful (feedback ratings).
4. The system stays online through the LDA of PA budget cycle on the free / near-free tier of Railway + Vercel + OpenAI.

Handoff risk is covered in **Doc 6 — Handoff**.

---

8. Related Documents

- **02 — System Architecture** — how the components fit together
- **03 — APIs** — every endpoint, request/response shape
- **04 — Data Modeling & DB Population** — schema, ERD, CSV import pipeline, data analytics
- **05 — Deployment** — Railway + Vercel setup, ongoing management
- **06 — Handoff** — non-technical owner's guide